

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ЛАБОРАТОРНАЯ РАБОТА №3
по дисциплине «Алгоритмы и структуры данных»
Тема: «Реализация и исследование структуры данных Bore»

Студентка гр. 4342

Киселева А.Д.

Преподаватель

Иванов Д.В.

Санкт-Петербург

2025

СОДЕРЖАНИЕ

1.	Цель работы	3
2.	Задание	4
3.	Теоретический минимум	6
4.	Выполнение работы	11
4.1.	Описание структуры кода	11
4.2.	Описание пайплайна	13
5.	Исследование	14
6.	Вывод	15
	Приложение А. Код программы	16
	Приложение Б. Тестирование	21

1. ЦЕЛЬ РАБОТЫ

Реализовать структуру данных *Rope* для работы с длинными строками, содержащую методы конкатенации с другой веревкой, разбиения веревки на две по индексу, получения символа по индексу, вставки строки в исходную строку, начиная с заданной позиции, удаления подстроки заданной длины из исходной строки, начиная с заданной позиции. Исследовать асимптотическую сложность функций и показать на графике зависимость времени работы от размера введенных данных.

2. ЗАДАНИЕ

Необходимо реализовать структуру данных *Rope* (верёвка). Она должна иметь следующие методы:

1. *concat* — конкатенация с другой верёвкой;
2. *split* — разбиение верёвки на две по индексу;
3. *index* — получение символа по индексу;
4. *insert* — вставка строки в исходную строку, начиная с заданной позиции.
5. *delete* — удаление подстроки заданной длины из исходной строки, начиная с заданной позиции.

Исследование

После успешного решения задачи в рамках курса проведите исследование данной структуры на различных размерах данных (10/1000/100000) и размерах листовых узлов, сравнив полученные результаты с теоретической оценкой (для лучшего, среднего и худшего случаев).

Примечание:

При решении задачи в курсе используйте максимальный размер листового узла, равный восьми.

Формат ввода

Первая строка содержит текст, состоящий из символов ASCII. На следующей строке дан индекс и через пробел — текст, который необходимо вставить по этому индексу. На третьей строке через пробел даны индекс начала и длина подстроки, которую нужно удалить из текста.

Формат вывода

Выводятся листья слева направо в формате “Leaf i: <строка, хранящаяся в листе>”, затем нелистовые узлы в порядке обхода в глубину в формате “Node i: <вес узла>”, затем выводится текст, хранящийся в верёвке. Такой вывод ожидается трижды: после построения верёвки, после вставки и после удаления.

3. ТЕОРЕТИЧЕСКИЙ МИНИМУМ

Структура данных *Rope* представляет собой двоичное сбалансированное дерево, где листовые узлы содержат небольшие фрагменты строки, внутренние узлы содержат вес (длину текста в левом поддереве), ссылки на левое и правое поддерева, высоту узла (для балансировки). *Rope* позволяет быстро выполнять операции со строками без необходимости копировать весь текст.

Дерево считается сбалансированным, если для каждого узла разность высот левого и правого поддеревьев (*balance factor*) находится в диапазоне $[-1, 1]$. При нарушении баланса выполняются повороты:

- Правый поворот: когда левое поддерево выше;
- Левый поворот: когда правое поддерево выше;
- Двойные повороты: когда требуется комбинация левого и правого поворотов.

Основные операции и их сложность:

1. Конкатенация (*concat*). Создаётся новый корневой узел, где левое и правое поддерева — это две исходные верёвки. Вес узла равен длине левого поддерева. После этого дерево балансируется. Операция выполняется за время $O(\log n)$, поскольку создание нового узла и балансировка дерева требуют обработки пути длиной $O(\log n)$ от корня до листьев.
2. Разбиение (*split*). Делит дерево по заданному индексу *idx* на два дерева: левое (до *idx*) и правое (после *idx*). Реализуется рекурсивно с учётом весов узлов. Операция имеет сложность $O(\log n)$, так как алгоритм проходит по пути длиной $O(\log n)$ в дереве, выполняя на каждом уровне постоянное количество операций.

3. Построение (*buildRope*). Исходная строка рекурсивно разбивается на куски примерно одинаковой длины, но не больше заданной, и собирается в сбалансированное дерево с помощью *concat*. Операция выполняется за время $O(n)$, поскольку каждый символ исходной строки обрабатывается ровно один раз при создании сбалансированной древовидной структуры.
4. Вставка (*insert*). Делает *split* по позиции вставки, строит *Rope* из вставляемой строки, и склеивает их. Операция требует $O(\log n + m)$ времени, где m - длина вставляемой строки, поскольку операция включает разбиение исходного дерева и построение нового поддерева из вставляемых данных.
5. Удаление (*deleteSubstr*). Делает два *split*: перед удаляемым диапазоном и после, а затем склеивает оставшиеся части. Операция выполняется за время $O(\log n)$, так как операция содержит два разбиения и одну конкатенацию, каждая из которых имеет логарифмическую сложность.

4. ВЫПОЛНЕНИЕ РАБОТЫ

4.1. Описание структуры кода

Реализована структура *Node*, хранящая информацию об отдельном узле *Rope*.

Поля структуры:

- *std::string str* — строка, хранящая подстроку (для листьев).
- *int weight* — вес узла, равный длине текста в левом поддереве.
- *int height* — высота узла в дереве, необходимая для балансировки.
- *Node* left* — указатель на левое поддерево.
- *Node* right* — указатель на правое поддерево.

Метод структуры:

- *Node(std::string s = "")* — конструктор, инициализирующий узел строкой *s*, весом 0, высотой 1 и нулевыми потомками.

Созданы следующие функции:

1. *int getHeight(Node* n)* — функция получения высоты узла.

Принимаемые аргументы:

Node n* — указатель на узел.

Возвращаемое значение:

int nodeHeight — высота узла, либо 0, если указатель равен *nullptr*.

2. *int getLength(Node* n)* — функция вычисления общей длины подстрок в дереве.

Принимаемые аргументы:

Node n* — указатель на корень поддерева.

Возвращаемое значение:

int length — суммарная длина строк, содержащихся в поддереве.

3. *void update(Node* n)* — функция пересчёта параметров узла. Обновляет *weight* (длину левого поддерева) и *height* (максимальную высоту поддеревьев).

Принимаемые аргументы:

Node n* — указатель на узел.

4. *int getBalance(Node* n)* — функция вычисления баланса узла.

Принимаемые аргументы:

Node n* — указатель на узел.

Возвращаемое значение:

int balanceFactor - Разность высот левого и правого поддеревьев.

5. *Node* rotateRight(Node* y)* — функция правого поворота поддерева.

Принимаемые аргументы:

Node y* — корень поддерева, требующего поворота.

Возвращаемое значение:

Node newRoot* — новый корень поддерева после поворота.

6. *Node* rotateLeft(Node* x)* — функция левого поворота поддерева.

Принимаемые аргументы:

Node x* — корень поддерева.

Возвращаемое значение:

Node newRoot* — новый корень после поворота.

7. *Node* balance(Node* node)* — функция балансировки дерева.

Принимаемые аргументы:

Node node* — узел, для которого требуется выравнивание.

Возвращаемое значение:

Node balancedNode* — сбалансированный узел.

8. *Node* concat(Node* left, Node* right)* — функция объединения двух поддеревьев.

Принимаемые аргументы:

Node left* — левое поддерево.

Node right* — правое поддерево.

Возвращаемое значение:

Node newRoot* — новый корень объединённого дерева.

9. *std::pair<Node*, Node*> split(Node* node, int idx)* — функция разделения дерева на две части.

Принимаемые аргументы:

Node node* — корень исходного дерева.

int idx — позиция разделения.

Возвращаемое значение:

std::pair<Node, Node*>* — два поддерева, полученных после разбиения.

10. *Node* buildRopeFromRange(const std::string& s, int l, int r)* — функция построения Rope из подстроки.

Принимаемые аргументы:

const std::string& s — исходная строка.

int l, r — границы подстроки.

Возвращаемое значение:

Node node* — корень дерева, построенного из диапазона [l, r).

11. *Node* buildRope(const std::string& s)* — управляющая функция построения Rope из всей строки.

Принимаемые аргументы:

const std::string& s — исходная строка.

Возвращаемое значение:

Node root* - корень построенного дерева.

12. *Node* insert(Node* root, int idx, const std::string& s)* — функция вставки подстроки.

Принимаемые аргументы:

Node root* — исходное дерево.

int idx — позиция вставки.

const std::string& s — вставляемая строка.

Возвращаемое значение:

Node newRoot* — новый корень дерева после вставки.

13. *Node* deleteSubstr(Node* root, int start, int len)* — функция удаления подстроки.

Принимаемые аргументы:

Node root* — исходное дерево.

int start — позиция начала удаления.

int len — длина удаляемой части.

Возвращаемое значение:

Node newRoot* — новый корень дерева после удаления.

14. *void collect(Node* n, std::string& res)* — функция обхода дерева для восстановления строки.

Принимаемые аргументы:

Node n* — корень дерева.

std::string& res — результирующая строка.

15. *void printLeaves(Node* n, int& id)* — функция вывода листьев (подстрок) дерева.

Принимаемые аргументы:

Node n* — корень дерева.

int& id — счётчик листьев.

16. *void printNodes(Node* n, int& id)* — функция вывода внутренних узлов дерева.

Принимаемые аргументы:

Node n* — корень дерева.

int& id — счётчик узлов.

17. *void printRope(Node* root)* — функция визуализации *Rope*. Выводит все листья, узлы и итоговую строку.

Принимаемые аргументы:

Node root* — корень дерева.

18. *int toInt(const std::string& s)* — функция преобразования строки в число.

Принимаемые аргументы:

const std::string& s — строка с числом.

Возвращаемое значение:

int value — целое число, представленное в строке.

19. *int main()* — основная функция программы. Считывает исходную строку, позицию и строку для вставки, параметры удаления. Затем строит *Rope*, выполняет вставку и удаление и выводит их результаты.

4.2. Описание пайплайна

Считывание данных происходит из стандартного потока ввода *stdin* с помощью функции *std::getline(std::cin, ...)*. Первая строка вводится в переменную *text* и содержит исходную строку, из которой будет строиться структура *Rope*. Вторая строка *line2* содержит позицию вставки и строку для вставки; она обрабатывается с помощью *find(' ')* для разделения позиции и текста, после чего числовое значение приводится к типу *int* функцией *toInt()*. Третья строка *line3* аналогично разделяется на позицию начала удаления и длину удаляемой подстроки, которые также приводятся к целым числам.

Предобработка данных заключается в корректном извлечении числовых параметров из строк ввода и подготовке исходной строки для построения *Rope*.

Основная работа с данными выполняется в функциях, связанных с построением и модификацией *Rope*:

- Функция *buildRope()* вызывает рекурсивную функцию *buildRopeFromRange()* для построения *Rope*-дерева из исходной строки.
- Функция *insert()* вставляет новую подстроку в заданную позицию, используя *split()* для разбиения *Rope* и *concat()* для объединения узлов.
- Функция *deleteSubstr()* удаляет подстроку заданной длины, также применяя *split()* и *concat()* для корректного обновления структуры.

Постобработка данных заключается в сборке окончательной строки из Rope при помощи функции *collect()*, а также в выводе состояния структуры (листов и узлов) через *printLeaves()* и *printNodes()*.

Вывод данных производится функцией *printRope()*, которая отображает текущее состояние Rope после каждого этапа (построение, вставка, удаление). Каждое дерево выводится в консоль с указанием листьев, узлов и финальной строки, формируемой из Rope.

ИССЛЕДОВАНИЕ

Для оценки временной сложности операций структуры данных Rope было проведено исследование времени выполнения функций вставки, удаления, поиска символа по индексу, конкатенации и разбиения при различных объёмах данных (от 100 до 1000000 узлов). По результатам эксперимента построены графики зависимости времени выполнения от количества узлов (см. рис. 1). Время выполнения операций *insert*, *delete*, *split* и *concat* растёт примерно логарифмически от размера данных, что соответствует теоретической сложности $O(\log n)$ для сбалансированных структур дерева.

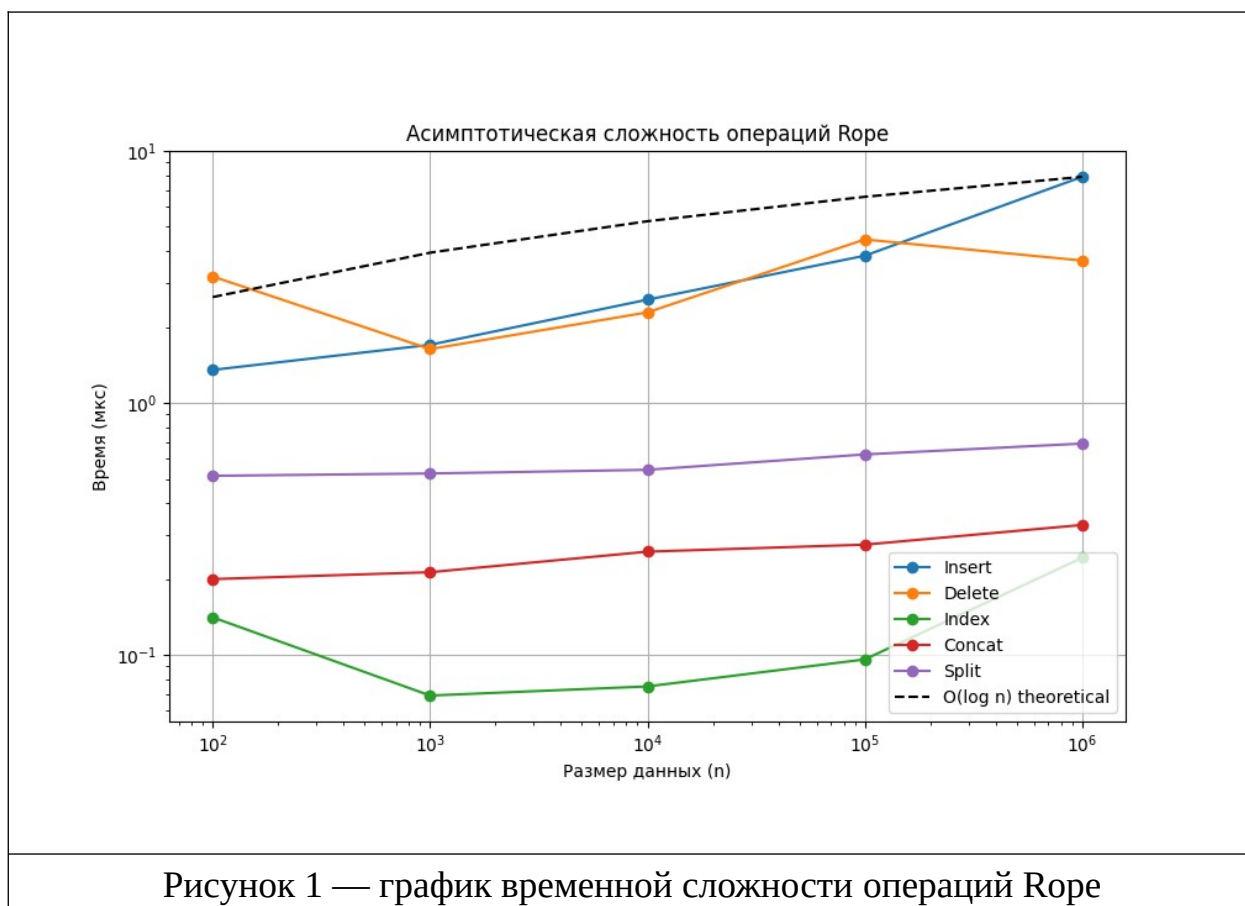


Рисунок 1 — график временной сложности операций Rope

6. ВЫВОД

В результате выполнения лабораторной работы был изучен и реализован алгоритм работы со структурой данных Bore, предназначенной для эффективного редактирования длинных строк. В процессе выполнения были реализованы основные операции над структурой: построение Bore из исходной строки, вставка подстроки в заданную позицию, удаление подстроки, а также балансировка дерева для поддержания эффективности операций. Были изучены принципы построения сбалансированных деревьев, особенности хранения строк в виде Bore и методы оптимизации работы со строками большой длины. Проведено исследование структуры на корректность выполнения операций вставки и удаления. Результатом стало создание программы, демонстрирующей эффективность Bore при работе с изменяемыми строками и обеспечивающей наглядное представление внутреннего устройства структуры.

ПРИЛОЖЕНИЕ А

КОД ПРОГРАММЫ

```
#include <iostream>
#include <vector>

const int MAX_LEAF = 8;

struct Node{
    std::string str;
    int weight;
    int height;
    Node* left;
    Node* right;

    Node(std::string s = ""):str(s), weight(0), height(1),
    left(nullptr), right(nullptr){}
};

int getHeight(Node* n){
    return n ? n->height : 0;
}

int getLength(Node* n){
    if (!n) return 0;
    if (!n->left && !n->right) return (int)n->str.size();
    return n->weight + getLength(n->right);
}

void update(Node* n){
    if (!n) return;
    n->weight = n->left ? getLength(n->left) : 0;
    int lh = getHeight(n->left);
    int rh = getHeight(n->right);
    n->height = (lh > rh ? lh : rh) + 1;
}

int getBalance(Node* n){
    return n ? getHeight(n->left) - getHeight(n->right) : 0;
}

Node* rotateRight(Node* y){
    Node* x = y->left;
    Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    update(y);
    update(x);
    return x;
}

Node* rotateLeft(Node* x){
```



```

    Node* y = x->right;
    Node* T2 = y->left;

    y->left = x;
    x->right = T2;

    update(x);
    update(y);
    return y;
}

Node* balance(Node* node){
    if (!node) return node;
    update(node);
    int b = getBalance(node);

    if (b > 1) {
        if (getBalance(node->left) < 0)
            node->left = rotateLeft(node->left);
        return rotateRight(node);
    }
    if (b < -1) {
        if (getBalance(node->right) > 0)
            node->right = rotateRight(node->right);
        return rotateLeft(node);
    }
    return node;
}

Node* concat(Node* left, Node* right){
    if (!left) return right;
    if (!right) return left;
    Node* root = new Node();
    root->left = left;
    root->right = right;
    update(root);
    return balance(root);
}

std::pair<Node*, Node*> split(Node* node, int idx){
    if (!node) return {nullptr, nullptr};
    update(node);

    if (!node->left && !node->right){
        int sz = (int)node->str.size();
        if (idx >= sz)
            return {node, nullptr};
        if (idx <= 0)
            return {nullptr, node};
        std::string l = node->str.substr(0, idx);
        std::string r = node->str.substr(idx);
        return {new Node(l), new Node(r)};
    }

    if (idx < node->weight){
        auto p = split(node->left, idx);
        Node* newRight = concat(p.second, node->right);
    }
}

```

```

        return {p.first, newRight};
    } else {
        auto p = split(node->right, idx - node->weight);
        Node* newLeft = concat(node->left, p.first);
        return {newLeft, p.second};
    }
}

Node* buildRopeFromRange(const std::string& s, int l, int r){
    int len = r - l;
    if (len <= MAX_LEAF) {
        return new Node(s.substr(l, len));
    }
    int mid = l + len / 2;
    Node* left = buildRopeFromRange(s, l, mid);
    Node* right = buildRopeFromRange(s, mid, r);
    return concat(left, right);
}

Node* buildRope(const std::string& s){
    if (s.empty()) return nullptr;
    return buildRopeFromRange(s, 0, (int)s.size());
}

Node* insert(Node* root, int idx, const std::string& s){
    auto p = split(root, idx);
    Node* mid = buildRope(s);
    return concat(concat(p.first, mid), p.second);
}

Node* deleteSubstr(Node* root, int start, int len){
    auto p1 = split(root, start);
    auto p2 = split(p1.second, len);
    return concat(p1.first, p2.second);
}

void collect(Node* n, std::string& res){
    if (!n) return;
    if (!n->left && !n->right) res += n->str;
    else {
        collect(n->left, res);
        collect(n->right, res);
    }
}

void printLeaves(Node* n, int &id){
    if (!n) return;
    if (!n->left && !n->right) {
        std::cout << "Leaf " << id++ << ": " << n->str << "\n";
    } else {
        printLeaves(n->left, id);
        printLeaves(n->right, id);
    }
}

void printNodes(Node* n, int& id){
    if (!n || (!n->left && !n->right)) return;

```

```

        std::cout << "Node " << id++ << ": " << n->weight << "\n";
        printNodes(n->left, id);
        printNodes(n->right, id);
    }

void printRope(Node* root){
    int id = 1;
    printLeaves(root, id);
    id = 1;
    printNodes(root, id);
    std::string result;
    collect(root, result);
    std::cout << result << "\n";
}

int toInt(const std::string& s){
    int sign = 1;
    int i = 0;
    if (s.size() > 0 && s[0] == '-') { sign = -1; i = 1; }
    int val = 0;
    for (; i < (int)s.size(); ++i) {
        char c = s[i];
        if (c < '0' || c > '9') break;
        val = val * 10 + (c - '0');
    }
    return val * sign;
}

int main(){
    std::string text;
    std::getline(std::cin, text);

    std::string line2;
    std::getline(std::cin, line2);
    int pos = (int)line2.find(' ');
    int insertPos = 0;
    std::string insertStr = "";
    if (pos == (int)std::string::npos) {
        insertPos = toInt(line2);
        insertStr = "";
    } else {
        insertPos = toInt(line2.substr(0, pos));
        insertStr = line2.substr(pos + 1);
    }

    std::string line3;
    std::getline(std::cin, line3);
    int pos3 = (int)line3.find(' ');
    int delPos = 0, delLen = 0;
    if (pos3 == (int)std::string::npos) {
        delPos = toInt(line3);
        delLen = 0;
    } else {
        delPos = toInt(line3.substr(0, pos3));
        delLen = toInt(line3.substr(pos3 + 1));
    }
}

```

```
Node* rope = buildRope(text);
printRope(rope);

rope = insert(rope, insertPos, insertStr);
printRope(rope);

rope = deleteSubstr(rope, delPos, delLen);
printRope(rope);

return 0;
}
```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ

```
#include <gtest/gtest.h>
#include "../main.cpp"

TEST(RopeConstruction, BuildFromEmptyString) {
    Node* rope = buildRope("");
    EXPECT_EQ(rope, nullptr);
}

TEST(RopeConstruction, BuildFromShortString) {
    Node* rope = buildRope("Hello");
    std::string result;
    collect(rope, result);
    EXPECT_EQ(result, "Hello");
}

TEST(RopeConstruction, BuildFromLongString) {
    std::string longString(500, 'X');
    Node* rope = buildRope(longString);
    std::string result;
    collect(rope, result);
    EXPECT_EQ(result, longString);
}

TEST(RopeConcat, ConcatTwoNonEmptyRopes) {
    Node* rope1 = buildRope("Hello");
    Node* rope2 = buildRope("World");
    Node* result = concat(rope1, rope2);

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "HelloWorld");
}

TEST(RopeConcat, ConcatWithEmptyLeft) {
    Node* rope1 = buildRope("");
    Node* rope2 = buildRope("World");
    Node* result = concat(rope1, rope2);

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "World");
}

TEST(RopeConcat, ConcatWithEmptyRight) {
    Node* rope1 = buildRope("Hello");
    Node* rope2 = buildRope("");
    Node* result = concat(rope1, rope2);

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "Hello");
}
```

```

TEST(RopeSplit, SplitAtBeginning) {
    Node* rope = buildRope("Hello World");
    auto [left, right] = split(rope, 0);

    std::string leftStr, rightStr;
    collect(left, leftStr);
    collect(right, rightStr);

    EXPECT_EQ(leftStr, "");
    EXPECT_EQ(rightStr, "Hello World");
}

TEST(RopeSplit, SplitAtEnd) {
    Node* rope = buildRope("Hello World");
    auto [left, right] = split(rope, 11);

    std::string leftStr, rightStr;
    collect(left, leftStr);
    collect(right, rightStr);

    EXPECT_EQ(leftStr, "Hello World");
    EXPECT_EQ(rightStr, "");
}

TEST(RopeSplit, SplitInMiddle) {
    Node* rope = buildRope("Hello World");
    auto [left, right] = split(rope, 5);

    std::string leftStr, rightStr;
    collect(left, leftStr);
    collect(right, rightStr);

    EXPECT_EQ(leftStr, "Hello");
    EXPECT_EQ(rightStr, " World");
}

TEST(RopeSplit, SplitSingleCharacter) {
    Node* rope = buildRope("A");
    auto [left, right] = split(rope, 1);

    std::string leftStr, rightStr;
    collect(left, leftStr);
    collect(right, rightStr);

    EXPECT_EQ(leftStr, "A");
    EXPECT_EQ(rightStr, "");
}

TEST(RopeInsert, InsertAtBeginning) {
    Node* rope = buildRope("World");
    Node* result = insert(rope, 0, "Hello ");

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "Hello World");
}

```

```

TEST(RopeInsert, InsertAtEnd) {
    Node* rope = buildRope("Hello");
    Node* result = insert(rope, 5, " World");

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "Hello World");
}

TEST(RopeInsert, InsertInMiddle) {
    Node* rope = buildRope("Hello World");
    Node* result = insert(rope, 5, " Beautiful");

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "Hello Beautiful World");
}

TEST(RopeInsert, InsertEmptyString) {
    Node* rope = buildRope("Hello World");
    Node* result = insert(rope, 5, "");

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "Hello World");
}

TEST(RopeDelete, DeleteFromBeginning) {
    Node* rope = buildRope("Hello World");
    Node* result = deleteSubstr(rope, 0, 6);

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "World");
}

TEST(RopeDelete, DeleteFromEnd) {
    Node* rope = buildRope("Hello World");
    Node* result = deleteSubstr(rope, 5, 6);

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "Hello");
}

TEST(RopeDelete, DeleteFromMiddle) {
    Node* rope = buildRope("Hello World");
    Node* result = deleteSubstr(rope, 5, 1);

    std::string strResult;
    collect(result, strResult);
    EXPECT_EQ(strResult, "HelloWorld");
}

TEST(RopeDelete, DeleteZeroCharacters) {
    Node* rope = buildRope("Hello World");
    Node* result = deleteSubstr(rope, 5, 0);
}

```

```

        std::string strResult;
        collect(result, strResult);
        EXPECT_EQ(strResult, "Hello World");
    }

    TEST(RopeDelete, DeleteEntireString) {
        Node* rope = buildRope("Hello World");
        Node* result = deleteSubstr(rope, 0, 11);

        std::string strResult;
        collect(result, strResult);
        EXPECT_EQ(strResult, "");
    }

    TEST(RopeEdgeCases, VeryLongString) {
        std::string longStr(10000, 'A');
        Node* rope = buildRope(longStr);
        std::string result;
        collect(rope, result);
        EXPECT_EQ(result, longStr);
    }

    TEST(RopeEdgeCases, SeveralOperations) {
        Node* rope = buildRope("Start");
        rope = insert(rope, 5, " Middle");
        rope = insert(rope, 0, "Prefix ");
        rope = deleteSubstr(rope, 6, 7);

        std::string result;
        collect(rope, result);
        EXPECT_EQ(result, "PrefixMiddle");
    }

    TEST(RopeTraversal, CollectEmptyRope) {
        Node* rope = buildRope("");
        std::string result;
        collect(rope, result);
        EXPECT_EQ(result, "");
    }

    int main(int argc, char **argv) {
        ::testing::InitGoogleTest(&argc, argv);
        return RUN_ALL_TESTS();
    }

```